

FreshLens

AI Powered Freshness Monitoring System for Small Scale Retailers

Test Plan Report

Group 21

PID 5

Contents

1	Evaluation Mission and Test Motivation	2
1.1	Out of Scope	2
1.2	Dependencies	3
1.3	Assumptions	3
1.4	Constraints	3
2	Target Test Items	3
3	Test Layering and Approach	4
3.1	Test Layering Table	4
3.2	Two-Dimensional Coverage Definition	5
4	Testing Techniques and Types	5
4.0.1	Data and Database Integrity Testing	5
4.0.2	Function Testing	6
4.0.3	User Interface Testing	6
4.0.4	Load and Performance Testing	6
4.0.5	Security and Access Control Testing	7
5	Evidence, Reporting, and Deliverables	7
5.1	Evidence Retention Mechanics	7
5.2	Explicit Rules on Credentials	8
5.3	Calling Out Weakest Areas	8
5.4	Test Evaluation Summaries	8
6	Detailed Test Cases	8
6.1	Backend API Test Case: Scan Upload Workflow (API-01)	8
6.2	Backend API Test Case: Sales Idempotency (API-02)	9
6.3	Database Security Test Case: Tenant Boundary Isolation (SEC-01)	9
6.4	ML Pipeline Test Case: Identity Confidence Threshold (ML-01)	10
6.5	UI Test Case: Admin Dashboard Analytics Rendering (UI-01)	10
7	Appendix: Implemented Test Inventory	10
7.1	Backend API Tests (apps/api/tests/)	10
7.2	Machine Learning Worker Tests (packages/ml/tests/)	11
7.3	Frontend & Mobile Logic Tests (apps/web/ & apps/mobile/)	11
7.4	Database Security Tests (infra/db/tests/)	11
8	References	12

1 Evaluation Mission and Test Motivation

Hey everyone! This Master Test Plan report covers our overall approach to testing the FreshLens system. Since FreshLens is our core SaaS platform for grocery retailers (and our main project for CS3203), we really needed to make sure it works as expected under real-world conditions. The main mission here is to provide a reliable, easy-to-use, and highly secure online app for tracking inventory scans and freshness. To give our end-users the best experience, we realized we had to design and test this system pretty thoroughly across the board.

Since this is a web and mobile application handling people's business data, it needs to be spot on. Even a single bug or a small security loophole in the database could be a disaster for a vendor's inventory. Our user interfaces and the backend logic have to be solid to deliver exactly what the vendors expect. Since our requirements explicitly outline multi-tenant data isolation, we had to put together a well-organized testing process to verify that the system actually does what it's supposed to do.

Our system is built using a modern architecture—mostly FastAPI for the backend, Next.js for the web admin, and Expo for the mobile side. So our testing plan has to cover all these different moving parts.

The main objectives of our testing process are:

- **Finding bugs:** We want to catch as many bugs as possible and fix them before we actually deploy this to our test users. We're doing a special search specifically for data leaks to make sure vendor CVs (like their company details) and inventory are completely confidential.
- **Fixing design problems:** We need to find important problems in our design and implementation that might pose a threat to the quality of the system. For example, if our YOLO models are too slow, that's a problem we need to correct before the final delivery.
- **Risk mitigation:** We're identifying risks associated with the project implementation. Things like failing user acceptance because the mobile camera lags. We test and refactor these before they happen by getting feedback early on.
- **Quality standards:** The system has to keep up with existing standards. To achieve this, we're doing performance testing along with user acceptance testing to check if we're hitting our goals.
- **Requirements check:** The system simply has to meet its requirements. A proper testing process helps us verify that we are actually building the functional and non-functional requirements we agreed upon, rather than wasting time on useless features.
- **Legal/Process mandates:** We have to make sure we aren't violating any data privacy rules, so we're testing the system's legal and security boundaries. Every violation needs to be fixed before deployment.

1.1 Out of Scope

To keep our testing efforts focused on code our team actually wrote, the following areas are explicitly **out of scope** for this test plan:

- **Supabase Authentication Infrastructure:** We test our parsing of JWTs, but we do not test the internal uptime, password hashing algorithms, or email delivery systems of Supabase itself.
- **R2 / Cloud Storage Provider Internals:** We mock storage interactions or assume the bucket is available. The internal durability and replication of Cloudflare R2 are not tested by us.
- **GitHub Actions Engine:** We rely on GitHub Actions for our CI pipeline, but we assume the underlying runner infrastructure is stable.

1.2 Dependencies

Our testing relies heavily on the following external and internal dependencies to function correctly:

- **Working Test Accounts:** We require pre-seeded, active test accounts mapped to specific roles (`vendor`, `platform_admin`) and tenants to perform access control testing.
- **CI Pipeline Uptime:** Automated tests require GitHub Actions to remain online.
- **External Services:** Supabase endpoints (for JWKS retrieval) must stay reachable during integration test runs.

1.3 Assumptions

We are taking the following conditions as given during this testing phase:

- **Stable Requirements:** The core functional requirements for ML inference and tenant isolation are stable and won't undergo massive refactors late in the cycle.
- **Finalized RLS Policies:** The Postgres Row Level Security logic implemented in our raw SQL migrations is considered structurally finalized.
- **Destructible Test Data:** Any data residing in the test environment databases can be freely created, corrupted, and destroyed by automated scripts without affecting stakeholders.

1.4 Constraints

The following limits impact the extent of our testing effort:

- **No Dedicated QA Team:** Testing is performed entirely by the development team (us).
- **No Prod-Matching Environment:** Due to budget constraints, our load testing environments (local Docker stacks) do not have the same CPU/RAM specifications as a true production cloud cluster.
- **Time-Boxed Schedule:** Comprehensive testing is strictly time-boxed by our university project deadlines (M4, M5, M6).

2 Target Test Items

Rather than a simple list, we've broken down our target test items into a matrix that highlights the specific component and the main risk associated with it if testing fails.

Target Component	Description	Main Risk if untested/failed
User Interfaces (Mobile & Web)	The Next.js admin dashboard and React Native Expo scanning app.	The vendor cannot upload scans due to poor network logic, or the admin sees broken analytics charts.
Data Models & Database (Postgres)	The core relational schema and RLS policies (Postgres 16).	Catastrophic cross-tenant data leakage. A vendor sees another vendor's stock.
Functions (FastAPI & ML Worker)	The backend endpoints (<code>/api/v1/sales</code>) and the Celery ML tasks.	Race conditions lead to double stock deduction, or the ML model returns a false positive on rotten produce.
Event / Background Pipeline	The Celery and Redis queuing system managing asynchronous jobs.	Tasks are dropped silently during a worker node crash, resulting in lost inventory scans.
Release / CI Pipeline	The GitHub actions workflow itself that packages the builds.	A bad, failing build is accidentally shipped to the main branch or production.

3 Test Layering and Approach

We utilize a layered approach to testing, ensuring that fast, cheap tests run frequently, while slower, expensive tests run at critical milestones.

3.1 Test Layering Table

Test Layer	Focus Area	CI Execution Timing
Unit	Individual functions, Pydantic schemas, UI utilities (e.g., chunked storage parser).	Every commit and PR.
Integration	Database connections, FastAPI endpoint logic interacting with the database.	Every PR before merge.
API	Contract testing (checking requests/responses against OpenAPI spec).	Every PR before merge.
Interface (UI)	Component rendering and form state logic in React/React Native.	Every PR before merge.
Cross-device	Mobile layout rendering on iOS vs Android (Expo).	Before major releases/milestones.

Test Layer	Focus Area	CI Execution Timing
Non-functional	Load testing, RLS isolation security checks, performance profiling.	RLS on every PR; Load testing before release.
Manual	Camera access workflows, exploratory testing, UX evaluations.	Pre-deployment staging review.

3.2 Two-Dimensional Coverage Definition

Coverage Dimension	Definition	Tracking Method	Caveat
Requirement Coverage	Ensuring every business rule (like idempotent deductions) is explicitly validated by a test case.	Generated directly from the pytest suites matching function names to requirements.	Cannot guarantee a requirement is implemented <i>efficiently</i> .
Code Coverage	The percentage of lines of code executed during the automated test suites.	Reported per-service via <code>pytest-cov</code> and Jest coverage tools.	100% coverage does not guarantee correctness ; it only proves the line was executed.

4 Testing Techniques and Types

4.0.1 Data and Database Integrity Testing

Attribute	Details
Technique Objective:	Our database creation is done using SQL migrations. We need to test this automatically by throwing incorrect data and unauthorized queries at it.
Technique:	Invoke every entity using test scripts. Check that correct data is retrieved (e.g., Tenant A only sees Tenant A's data).
Oracles:	Test access by injecting data through code, checking behavior, and asserting retrieved data isn't erroneous.
Required Tools:	PostgreSQL 16, Pytest framework, custom SQL isolation scripts (<code>rls_isolation.sql</code>).

Attribute	Details
Success Criteria:	Every test is guaranteed to produce a reasonable outcome, proving RLS bounds are perfectly respected.

4.0.2 Function Testing

Attribute	Details
Technique Objective:	Exercise the target functionality, including data entry and processing, to log the behavior and check requirement fulfillment.
Technique:	Execute each use-case scenario's individual flow using valid and invalid data via Pytest. Verify appropriate error messages and business rules.
Oracles:	Create inputs, determine expected outputs based on OpenAPI specs, and assert the actual HTTP outcome.
Required Tools:	Pytest framework, FastAPI TestClient.
Success Criteria:	All tests execute properly without errors. 201 Created and 409 Conflict logic works flawlessly.

4.0.3 User Interface Testing

Attribute	Details
Technique Objective:	Observe rendering with minimum memory, high UX, platform independence, and smooth form submission workflows.
Technique:	Check whether complex logic (like chunked storage uploads) works properly under throttled network settings.
Oracles:	Automate testing for UI utilities (<code>claims.test.ts</code>), while relying on manual walkthroughs for complex gestures.
Required Tools:	Jest for React testing, ESLint, browser dev tools.
Success Criteria:	All routes resolve successfully, zero broken components, no memory leak warnings in console.

4.0.4 Load and Performance Testing

Attribute	Details
Technique Objective:	Exercise transactions under normal, worst-case, and concurrent user workloads to observe system performance degradation.
Technique:	Run loops in test scripts to simultaneously hit the API, increasing transaction volume to simulate spiky traffic.
Oracles:	Code must be automated to apply the load, monitoring CPU and database connection pool saturation.
Required Tools:	Custom async Python scripts and Pytest concurrency fixtures.
Success Criteria:	The system gracefully degrades or queues requests rather than dropping them or returning 500 errors.

4.0.5 Security and Access Control Testing

Attribute	Details
Technique Objective:	Ensure an actor can access only functions they have permissions for, and unauthenticated users are entirely blocked.
Technique:	Create tests for each user type (vendor , platform_admin). Verify forbidden messages when attempting unauthorized access.
Oracles:	Fully automated testing using Pytest to assert HTTP 401 Unauthorized and 403 Forbidden response codes.
Required Tools:	Pytest, PyJWT for generating mock tokens.
Success Criteria:	System rigidly enforces privileges; access denied messages are given strictly and correctly.

5 Evidence, Reporting, and Deliverables

We have established strict mechanics for how test evidence is captured and retained to ensure maximum transparency.

5.1 Evidence Retention Mechanics

- **Machine-Readable Result Files:** All test runs produce XML/JSON output files (like JUnit XML) that are archived as artifacts in GitHub Actions for historic trend analysis.

- **Raw Request Logs:** During load testing, we do not simply retain the “summary” report. Raw HTTP request logs (including latency and status codes) are retained in storage to allow for deep-dive debugging of latency spikes.
- **UI Test Failures:** (Future implementation) If a UI test fails, the CI pipeline will capture a screenshot/video of the headless browser state to immediately highlight the visual error.

5.2 Explicit Rules on Credentials

Strict Rule: No production or staging credentials, API keys, or raw passwords will ever be committed to the repository for testing purposes.

- All test suites must rely on mock generation, factory fakes, or rely on `.env.test` templates with blank fields. Real secrets are injected only at runtime by the CI/CD secure vault.

5.3 Calling Out Weakest Areas

Rather than hiding behind a blended coverage percentage (e.g., “The project has 85% coverage”), our test evaluation reports explicitly name the “weakest areas” that lack coverage. This ensures we are always aware of technical debt (for instance, calling out that mobile offline caching currently lacks automated coverage).

5.4 Test Evaluation Summaries

- **Test Logs:** These are the detailed logs that give the outcome of every test written for the functionalities. If even a single test log gives an output as a failure in our GitHub Actions, that functionality is rechecked and all tests are run again.
- **Code Inspection Tool:** Code inspection details are gotten after running ESLint and TypeScript checkers in the IDE. This notifies us about spelling mistakes, empty tags, and unused items.
- **Database Performance Statistics:** This is a very good method to check database performance. It checks memory usage and size.

6 Detailed Test Cases

To provide a much more thorough explanation of our testing workflow, we have detailed our most critical test cases below. Rather than a brief overview, these describe exactly why and how we test each functional component of the FreshLens system to ensure it holds up to real-world vendor usage.

6.1 Backend API Test Case: Scan Upload Workflow (API-01)

- **Motivation:** Vendors operating in grocery stores often have terrible Wi-Fi or cellular connections. If they upload a large image of produce, it cannot time out and crash the application.
- **Pre-conditions:** The mobile app is authenticated with a valid `vendor` JWT token. The server is online and connected to the Redis broker.
- **Execution Steps:**

1. The vendor opens the camera within the Expo app.
 2. The vendor takes a high-resolution photo of a banana.
 3. The photo is chunked locally and transmitted via HTTP POST to `/api/v1/scans`.
- **Expected System Behavior:** The FastAPI backend should immediately capture the image byte chunks, save them to the R2 bucket, and spawn an asynchronous Celery task. Critically, it must immediately return a `202 Accepted` status code to the mobile app so the vendor isn't left waiting for the ML inference to finish.
 - **Status: PASS.** Verified via `test_scans.py`.

6.2 Backend API Test Case: Sales Idempotency (API-02)

- **Motivation:** If a vendor hits the “Submit Sale” button twice by accident, or if the network stutters and resends the request, we absolutely cannot deduct stock twice. This would ruin the vendor’s inventory records.
- **Pre-conditions:** The database has an existing inventory batch of tomatoes with a quantity greater than zero.
- **Execution Steps:**
 1. A script generates a unique `Idempotency-Key` header.
 2. The script fires two completely identical POST requests simultaneously to `/api/v1/sales`.
- **Expected System Behavior:** The backend must rely on a database transaction lock or caching layer. The very first request processed should succeed and return `201 Created`, deducting the stock exactly once. The second concurrent request should hit the idempotency lock and gracefully return a `409 Conflict`, without modifying the database.
- **Status: PASS.** Verified via `test_sales.py`.

6.3 Database Security Test Case: Tenant Boundary Isolation (SEC-01)

- **Motivation:** Since this is a multi-tenant SaaS application, multiple different grocery retailers are sharing the same database tables. If Tenant A can somehow view or edit Tenant B’s sales data, the entire platform’s security is compromised.
- **Pre-conditions:** The Postgres 16 database is seeded with data for both “Tenant A” and “Tenant B”. Row Level Security (RLS) is active.
- **Execution Steps:**
 1. An automated SQL script logs into the database using the restricted `freshlens_api_runtime` role.
 2. The script sets the local transaction context to Tenant A.
 3. The script attempts to run `SELECT * FROM scans WHERE tenant_id = 'Tenant_B_ID'`.
- **Expected System Behavior:** Instead of crashing, the Postgres RLS engine silently filters out the unauthorized rows, returning an empty list (0 rows) to Tenant A. The application backend is entirely shielded from accidentally leaking data.
- **Status: PASS.** Verified via `infra/db/tests/rls_isolation.sql`.

6.4 ML Pipeline Test Case: Identity Confidence Threshold (ML-01)

- **Motivation:** The YOLO object detection model is incredibly fast but can sometimes be overconfident when analyzing blurry images or items it has never seen before. We cannot let it misclassify a blurry eggplant as a cucumber and ruin inventory counts.
- **Pre-conditions:** The YOLO26n-c1s model is loaded into the Celery worker memory.
- **Execution Steps:**
 1. A known ambiguous or blurry image is passed directly to the inference pipeline via the testing framework.
- **Expected System Behavior:** The model evaluates the image. Even if its top prediction is “cucumber”, it must check the confidence score. Because the confidence score is strictly enforced to be ≥ 0.75 for identity, the script must intercept a lower score (e.g., 0.60) and override the prediction, safely returning an `unknown` label instead of logging a false positive.
- **Status: PASS.** Verified via `test_identity.py`.

6.5 UI Test Case: Admin Dashboard Analytics Rendering (UI-01)

- **Motivation:** The platform administrator needs to view aggregate data. If the TypeScript data mapping logic fails when receiving an unexpected payload from the API, the entire React screen will crash (White Screen of Death).
- **Pre-conditions:** The user is logged in via Supabase with the `app_role` explicitly set to `platform_admin`.
- **Execution Steps:**
 1. The admin navigates to the core analytics dashboard URL.
 2. The page fires a fetch request for aggregate mapping data.
- **Expected System Behavior:** The Next.js frontend utilizes `admin-map.test.ts` logic to parse the arrays. It should map the data without throwing any TypeScript `undefined` errors and cleanly render the visualization chart, displaying aggregated cross-tenant metrics flawlessly.
- **Status: PASS.** Verified via Jest UI suites.

7 Appendix: Implemented Test Inventory

Here is the exhaustive inventory of all test suites currently implemented in our project. We wrote these to make sure our system is as robust as possible.

7.1 Backend API Tests (`apps/api/tests/`)

- `test_admin.py`
- `test_scans.py`
- `test_auth.py`
- `test_storage.py`
- `test_authorization.py`
- `test_config.py`
- `test_health.py`
- `test_sales.py`

- `test_tenant_context.py`

7.2 Machine Learning Worker Tests (`packages/ml/tests/`)

- `test_prepare_identity_dataset.py`
- `test_stub_classifier.py`
- `test_evaluate_freshness.py`
- `test_inventory_batch.py`
- `test_imagenet_produce.py`
- `test_prepare_freshness_dataset.py`
- `test_identity.py`
- `test_dataset_snapstock.py`

7.3 Frontend & Mobile Logic Tests (`apps/web/` & `apps/mobile/`)

- `claims.test.ts` (Web)
- `admin-map.test.ts` (Web)
- `claims.test.ts` (Mobile)
- `chunked-storage.test.ts` (Mobile)

7.4 Database Security Tests (`infra/db/tests/`)

- `rls_isolation.sql`
- `runtime_role.sql`

8 References

- FreshLens System Architecture Document (`docs/design/FreshLens-SAD.md`)
- FreshLens API Contract (`docs/api/v1/openapi.yaml`)
- Pytest Testing Framework available at <https://docs.pytest.org/>
- Jest Testing Framework available at <https://jestjs.io/>
- Supabase Authentication Architecture available at <https://supabase.com/docs/guides/auth>